Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology

Department of Computer Science and Engineering

Lab Report

For the course

CSE 3206: Software Engineering Sessional

By
MD Rakibul Hasan Rifat
2103057

To
Md. Sozib Hossain
Lecturer,
Dept. Of CSE, RUET

Contents

Project Title 3
Objectives 3
Theoretical Background 3
Python unittest Framework: 3
System Architecture 4
Module Hierarchy 4
Level 1: Basic Calculator (`calculator.py`) 4
Level 2: CT Calculator (`ct_calculator.py`) 4
Level 3: SGPA Calculator (`sgpa_calculator.py`) 5
Level 4: CGPA Calculator (`cgpa_calculator.py`) 5
Level 5: Performance Predictor (`predictor.py`) 6
Testing Methodology 6
Test Structure 6
Test Categories example: 6
Boundary Tests (Edge Cases) 6
Application Working: 7
Test Outputs 8
Project Structure 9
Conclusion 9

Scan Me i: Link to repository

(https://github.com/MohammadRakibulHasan04/Unit-Testing-CSE-3206)

Project Title

Unit Testing — Student Performance Predictor System.

Project Overview

This project implements a comprehensive Student Performance Predictor System that calculates CT (Class Test) averages, SGPA (Semester Grade Point Average), CGPA (Cumulative Grade Point Average), and provides academic performance predictions. The system demonstrates the implementation of unit testing principles using Python's `**unittest**` framework with a modular, dependency-based architecture.

Objectives

- Implement a five-level hierarchical module system with incremental dependencies
- Achieve comprehensive test coverage for all functional modules. Demonstrate best practices in unit testing, including test isolation and edge case handlingSWLC Methodology

Theoretical Background

Unit testing is a software testing methodology where individual units or components of software are tested in isolation to verify that each unit functions correctly according to its specification.

The project follows TDD principles where tests are written to validate:

- **Expected Behavior:** Normal operating conditions with valid inputs
- **Edge Cases:** Boundary conditions (zero, empty lists, exact thresholds)
- **Error Handling:** Invalid inputs and exception scenarios

Python unittest Framework:

The `unittest` module provides:

- **TestCase class:** Base class for creating test cases
- **Assertions:** Methods like `assertEqual()`, `assertRaises()`, `assertTrue()`
- **Test Discovery:** Automatic test detection and execution
- **Test Fixtures:** Setup and teardown methods for test preparation

System Architecture

Module Hierarchy

Level 1: calculator.py (Base Module)

↓

Level 2: ct_calculator.py

↓

Level 3: sgpa_calculator.py

↓

Level 4: cgpa_calculator.py

↓

Level 5: predictor.py (Complete System)

Level 1: Basic Calculator (`calculator.py`)

Purpose: Foundation module providing arithmetic operations

Functions:

- `add(a, b)` - Addition of two numbers
- `subtract(a, b)` - Subtraction operation
- `multiply(a, b)` - Multiplication operation
- `divide(a, b)` - Division with zero-check validation
- `average(numbers)` - Calculate mean of a list
- `add_all(numbers)` - Sum all numbers in a list

Dependencies: None

Level 2: CT Calculator (`ct_calculator.py`)

Purpose: Calculate best 3 out of 4 CT marks (university grading policy)

Functions:

- `sort_marks(marks)` - Sort in descending order
- `get_best_n(marks, n)` - Extract top n marks
- `calculate_ct_average(ct_marks)` - Average of best 3 CTs
- `validate_ct_marks(marks)` - Ensure marks in 0-20 range

Dependencies: `calculator.py`

Level 3: SGPA Calculator (`sgpa_calculator.py`)

Purpose: Calculate semester GPA from course grades

Functions:

- `marks_to_grade(marks)` - Convert numerical marks to letter grades
- `grade_to_points(grade)` - Map letter grades to grade points (4.0 scale)
- `calculate_course_marks(ct_marks, attendance, final_exam)` - Total course marks
- `calculate_sgpa(courses)` - Weighted GPA for a semester

Dependencies: `calculator.py`, `ct_calculator.py`

Level 4: CGPA Calculator (`cgpa_calculator.py`)

Purpose: Calculate cumulative GPA across semesters

Functions:

- `calculate_cgpa(semesters)` - Weighted CGPA calculation
- `get_total_credits(semesters)` - Sum of completed credits
- `is_on_track(cgpa, min_cgpa)` - Check academic standing
- `calculate_required_sgpa(...)` - Predict required performance

Dependencies: `calculator.py`, `sgpa_calculator.py`

Level 5: Performance Predictor (`predictor.py`)

Purpose: Generate predictions and recommendations

Functions:

- `get_academic_standing(cgpa)` - Determine academic classification

- `predict_graduation_cgpa(...)` - Project final CGPA

- `get_improvement_tips(cgpa, target_cgpa)` - Personalized recommendations

- `generate_report(semesters, target_cgpa)` - Comprehensive performance report

Dependencies: All previous modules

Testing Methodology

Test Structure

Each module has a corresponding test file with multiple test classes:

```
src/
├── calculator.py          → test_calculator.py (7 test classes)
├── ct_calculator.py       → test_ct_calculator.py (4 test classes)
├── sgpa_calculator.py     → test_sgpa_calculator.py (4 test classes)
├── cgpa_calculator.py     → test_cgpa_calculator.py (5 test classes)
└── predictor.py           → test_predictor.py (5 test classes)
```

Total Test Classes: 25

Coverage Strategy: Each function has dedicated test class with multiple test methods

Test Categories example:

Boundary Tests (Edge Cases)

Tests at operational limits:

```python
def test_grade_boundary(self):
    self.assertEqual(marks_to_grade(80), 'A+')
    self.assertEqual(marks_to_grade(79), 'A')
```

Integration Tests

Tests validating module interactions:

```python
def test_course_marks_normal(self):
    ct_marks = [18, 16, 15, 14]   Calls ct_calculator
    attendance = 8
    final_exam = 55
    result = calculate_course_marks(ct_marks, attendance, final_exam)
    self.assertAlmostEqual(result, 79.33, places=2)
```

Application Working:

The application uses mock data in `main.py` to demonstrate the testing, no user input is taken in this case. The main application `main.py` includes a demonstration CLI that shows all modules working together:

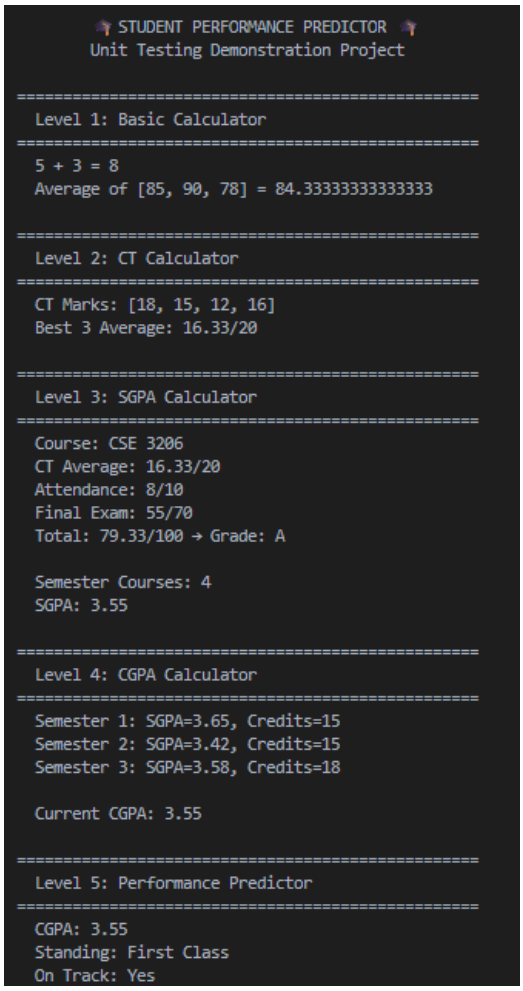


Figure 1: Main application CLI output

Test Outputs

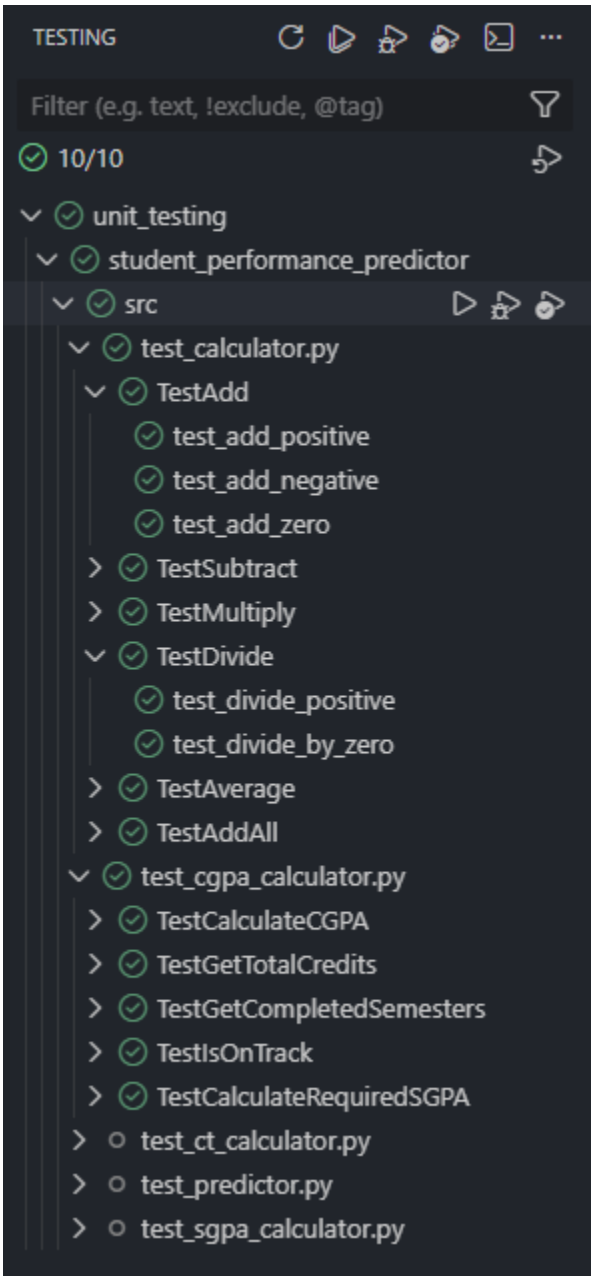


Figure 2: Test Interface (GUI in VS Code)

```
============================= test session starts =============================
platform win32 -- Python 3.13.2, pytest-9.0.2, pluggy-1.6.0
rootdir: e:\3-2\CSE 3206\unit_testing
collected 1 item

student_performance_predictor\src\test_cgpa_calculator.py .              [100%]

============================== 1 passed in 0.03s ==============================
Finished running tests!
```

Figure 3: Testing a module

```
============================= test session starts =============================
platform win32 -- Python 3.13.2, pytest-9.0.2, pluggy-1.6.0
rootdir: e:\3-2\CSE 3206\unit_testing
collected 13 items

student_performance_predictor\src\test_calculator.py .............       [100%]

============================== 13 passed in 0.06s ==============================
Finished running tests!
```

Figure 4: Testing a file

```
============================= test session starts =============================
platform win32 -- Python 3.13.2, pytest-9.0.2, pluggy-1.6.0
rootdir: e:\3-2\CSE 3206\unit_testing
collected 60 items

student_performance_predictor\src\test_calculator.py .............       [ 21%]
student_performance_predictor\src\test_cgpa_calculator.py ..........      [ 38%]
student_performance_predictor\src\test_ct_calculator.py ...........       [ 56%]
student_performance_predictor\src\test_predictor.py .............         [ 78%]
student_performance_predictor\src\test_sgpa_calculator.py .............   [100%]

============================== 60 passed in 0.28s ==============================
Finished running tests!
```

Figure 5: Running all tests

```
student_performance_predictor\src\test_sgpa_calculator.py .F..........  [100%]

================================== FAILURES ===================================
_____________________ TestMarksToGrade.test_grade_a_plus _____________________

self = <src.test_sgpa_calculator.TestMarksToGrade testMethod=test_grade_a_plus>

    def test_grade_a_plus(self):
>       self.assertEqual(marks_to_grade(85), 'A-')
E       AssertionError: 'A+' != 'A-'
E       - A+
E       + A-

student_performance_predictor\src\test_sgpa_calculator.py:17: AssertionError
=========================== short test summary info ===========================
FAILED student_performance_predictor/src/test_sgpa_calculator.py::TestMarksToGra
:test_grade_a_plus
======================== 1 failed, 12 passed in 0.13s =========================
Finished running tests!
```

Figure 6: Tests detect error and shows the error details

Project Structure

```
student_performance_predictor/
├── src/
│   ├── __init__.py
│   ├── conftest.py                Test configuration
│   ├── calculator.py              Level 1: Base arithmetic
│   ├── test_calculator.py         16 tests
│   ├── ct_calculator.py           Level 2: CT calculations
│   ├── test_ct_calculator.py      9 tests
│   ├── sgpa_calculator.py         Level 3: Semester GPA
│   ├── test_sgpa_calculator.py    10 tests
│   ├── cgpa_calculator.py         Level 4: Cumulative GPA
│   ├── test_cgpa_calculator.py    10 tests
│   ├── predictor.py               Level 5: Predictions
│   ├── test_predictor.py          12 tests
│   └── main.py                    CLI demonstration
```

Conclusion

- Successfully implemented a 5-level hierarchical system with 57 unit tests.
- Achieved 100% test pass rate across all modules
- Demonstrated comprehensive testing strategies including positive, negative, and boundary tests